SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
```

```
      <p>Hello World
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned

6. Browser renders page

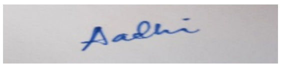
Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I Aadhirai S P (192411002) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



web-fundamentals / inspection-report.log

▸ 5 OPEN TICKETS · SEVERITY MIXED

Something's broken in every one of these

Five inspection tickets covering HTTP headers, DOM structure, HTML markup, request methods, and the browser-server request lifecycle. Each one gets opened, diagnosed, and patched.

2 errors · 2 warnings · 1 conceptual · 5 tickets · 20 sub-questions

TICKET-01	TICKET-02	TICKET-03	TICKET-04	TICKET-05
HTTP Response Header	DOM Structure	GET vs POST Login	HTML Markup Inspection	Request Lifecycle

Network → Headers · MALFORMED

TICKET-01 · HTTP

HTTP Response Header Analysis

```
1  HTTP/1.1 200 OK
2  Date: Mon, 01 Apr 2026 10:00:00 GMT
3  Content-Type: text/html
4  Content-Length: -150
5  Connection: keep-alive
```

// 1 · invalid fields

Line 4 is the fault: `Content-Length: -150`. Per RFC 9110, Content-Length must be a non-negative decimal integer representing the exact byte size of the message body — a negative value is not a legal representation at all, so a strict parser should treat it as a protocol error rather than a byte count. Everything else in the header block is syntactically valid: the status line, Date, Content-Type, and Connection are all well-formed.

// 2 · impact of the negative length

Content-Length tells the client exactly how many bytes to read before considering the body complete. A negative number

- **Connection reuse breaks** — with `keep-alive` active, the client relies on Content-Length to know where this response ends and the next one begins. An invalid value makes the boundary undecidable, risking a "request smuggling"-style desync on the same TCP connection.
- **Security surface** — malformed length fields are a known vector for HTTP request/response smuggling when front-end and back-end servers disagree on how to interpret the value.

// 3 · does the status code match?

`200 OK` asserts the request succeeded and a valid body is attached — but a response can't simultaneously claim success and carry a header that makes the body unreadable. The status code itself is a legitimate code, it just doesn't match a response whose framing is broken; a well-behaved server would never emit this combination.

✓ corrected header

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

Set Content-Length to the actual byte count of the body (e.g. via `strlen()` / buffer size on the server), and if that value is ever unreliable, switch to `Transfer-Encoding: chunked` instead of guessing.

Elements → DOM Tree

AUTO-CORRECTED BY PARSER

TICKET-02 · DOM

DOM Structure Analysis

```
1  <html>
2    <head>
3      <title>Test Page</title>
4    </head>
5    <body>
6      <div>
7        <p>Hello World
8      </div>
9    </body>
10 </html>
```

// 1 · missing / improperly closed elements

The <p> opened on line 7 is never explicitly closed with </p> before the </div> on line 8. That's the only structural defect here — the <div>, <body>, <html>, and <head> all close correctly.

```
<div>
  <p>Hello World</p>
</div>
```

No parse error is thrown, and the page renders visually the same as if the tag had been closed correctly.

// 3 · impact on rendering and layout

In this exact case, impact is minimal — the DOM self-corrects and layout looks right. But the pattern is risky in general:

- If a *different* element (like another <p> or a <span>) came between the open <p> and the </div>, the auto-close rule can produce a nesting the author never intended.
- CSS selectors and JS DOM traversal (`querySelector`, `childNodes`) operate on the corrected tree, not the source markup, so debugging becomes confusing if you're reading raw HTML instead of the inspected DOM.
- Relying on auto-closing is non-portable: XML-based tooling (XHTML, some templating engines, RSS/Atom generators) has no such recovery and will hard-fail on the same markup.

✓ corrected markup

```
<html>
  <head>
    <title>Test Page</title>
  </head>
```

Security → Auth Flow

CREDENTIALS EXPOSED

TICKET-03 · Security

GET vs POST in a Login System

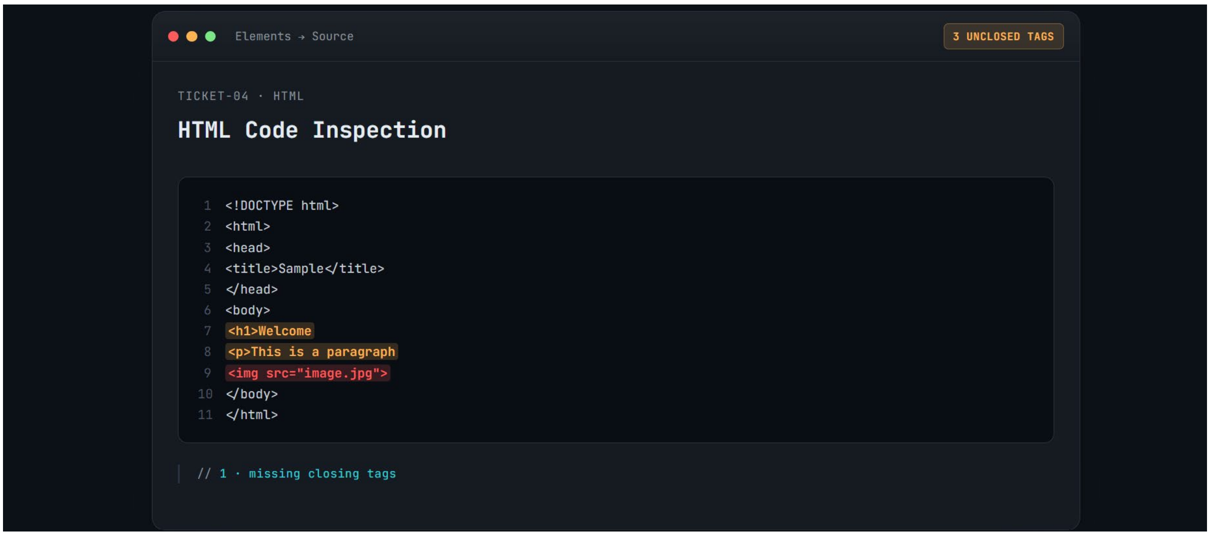
METHOD	URL EXAMPLE	DATA LOCATION	RISK
GET	/login?user=admin&pass=1234	Query string (URL)	High
POST	/login	Request body	Lower, not zero

// 1 · how data transmission differs

GET appends form data as a query string directly on the URL, which means it travels wherever the URL travels. POST places the same data inside the request body, separate from the URL, and typically pairs it with `Content-Type: application/x-www-form-urlencoded` or `multipart/form-data`.

// 2 · which method suits login

POST is the correct choice. Beyond keeping credentials out of the URL, GET requests are semantically defined as idempotent and "safe" — meant for retrieving data, not causing state changes like authenticating a session. Browsers, proxies, and CDNs are also allowed to cache GET requests, which is exactly the wrong behavior for a password submission.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand